The German International University for Applied Sciences

Faculty of Business Informatics

Software Engineering

Spring 2026

Submitted by:

Salma Walid 13007299 (Team Leader)

Mai Mohamed 13006213

Aya Hussein 13004531

Maram Maher 13006996

Mariam Mahmoud 14005095

Salma Eldesouky 14003143

Rawda Elmetwally 14003722

Farida Farag 13005412

Tutorial: 5

Submitted to: Hassan Osman

CampusCare
Software Requirements Specification (SRS)

1. Introduction

Purpose

This Software Requirements Specification (SRS) document provides a detailed description of the CampusCare mobile application, a Smart Facility Management System developed for the German International University (GIU).

The purpose of this document is to define the system objectives, architecture, scope, functional requirements, UML structure, ERD structure, and technical implementation details in order to guide development, deployment, testing, and future maintenance.

CampusCare was developed to solve inefficient communication between university community members and the Facility Management department. Maintenance issues are often reported informally which leads to delayed responses and poor issue tracking.

The application introduces a centralized digital solution that allows users to report issues quickly through their mobile devices while enabling the Facility Management team to manage, assign, monitor, and resolve issues efficiently.

Intended Audience

- Software developers implementing the system
- Course instructors and evaluators
- Business analysts and database designers
- Facility management stakeholders
- Future maintainers and contributors

Overview of the Software

CampusCare is a mobile application that digitizes campus maintenance workflows. The system enables students, academic staff, and university employees to report maintenance and infrastructure issues directly from their mobile devices.

- Upload issue photos
- Enter issue descriptions
- Specify issue locations
- Track issue status
- View issue details
- Assign workers to maintenance tasks
- Update issue statuses
- Upload completion photos
- Add worker comments

1.1 Product Vision & Scope

Product Vision

The vision of CampusCare is to provide a centralized, user-friendly, and efficient digital platform that improves communication between university community members and the Facility Management department.

- Reduce maintenance response time
- Improve transparency in maintenance operations
- Improve campus cleanliness and safety
- Allow users to track issue progress
- Enable efficient workforce management
- Maintain historical maintenance records

Scope

- Mobile application for students and staff
- Facility manager dashboard
- Worker dashboard
- Issue submission and tracking
- Role-based authentication and authorization
- Worker assignment functionality
- Issue status management
- Worker comments
- Completion photo uploads
- Cloud image storage
- PostgreSQL database using Supabase

1.2 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
PostgreSQL	Relational Database used through Supabase
Expo	React Native development framework

2. User Roles

2.1 Community Member

Description

Any student, staff member, or university employee who reports issues.

Responsibilities

- Report maintenance issues
- Provide accurate issue information
- Track issue status
- View submitted issues

Permissions

- Register account
- Login/logout
- Submit issues
- Upload issue photos
- View issue details

2.2 Facility Manager

Description

Supervises maintenance workflow and manages all issues.

Responsibilities

- Monitor all issues
- Assign issues to workers
- Track issue progress
- Close resolved issues

Permissions

- View all issues
- Assign workers
- Delete invalid issues
- Update issue statuses

2.3 Worker

Description

Responsible for resolving maintenance tasks.

Responsibilities

- View assigned issues
- Update issue progress
- Upload completion photos
- Add comments

Permissions

- View assigned issues
- Update statuses
- Upload completion photos
- Add comments

3. Functional Requirements

3.1 Authentication & Security

ID	Requirement	Priority
FR-18	The system must allow users to register accounts securely.	High
FR-19	The system must authenticate users using JWT tokens.	High
FR-20	The system must enforce role-based access control (RBAC).	High
FR-21	The system must support password reset functionality.	Medium

3.2 Community Member Functional Requirements

ID	Requirement	Priority
FR-22	The system must allow users to submit maintenance issues.	High
FR-23	The system must allow users to upload issue photos.	High
FR-24	The system must allow users to track issue status.	High
FR-25	The system must allow users to view submitted issues.	High

3.3 Facility Manager Functional Requirements

ID	Requirement	Priority
FR-26	The system must allow managers to assign workers.	High
FR-27	The system must allow managers to close resolved issues.	High
FR-28	The system must allow managers to delete invalid issues.	Medium

3.4 Worker Functional Requirements

ID	Requirement	Priority
FR-29	The system must allow workers to view assigned issues.	High
FR-30	The system must allow workers to upload completion photos.	Medium
FR-31	The system must allow workers to add comments.	Medium

4. Updated User Stories

Community Member

- As a user, I want to register securely so that I can access the system.
- As a user, I want to submit maintenance issues with descriptions and photos.
- As a user, I want to track submitted issues.

Worker

- As a worker, I want to view assigned issues.
- As a worker, I want to update issue statuses.
- As a worker, I want to upload completion photos.

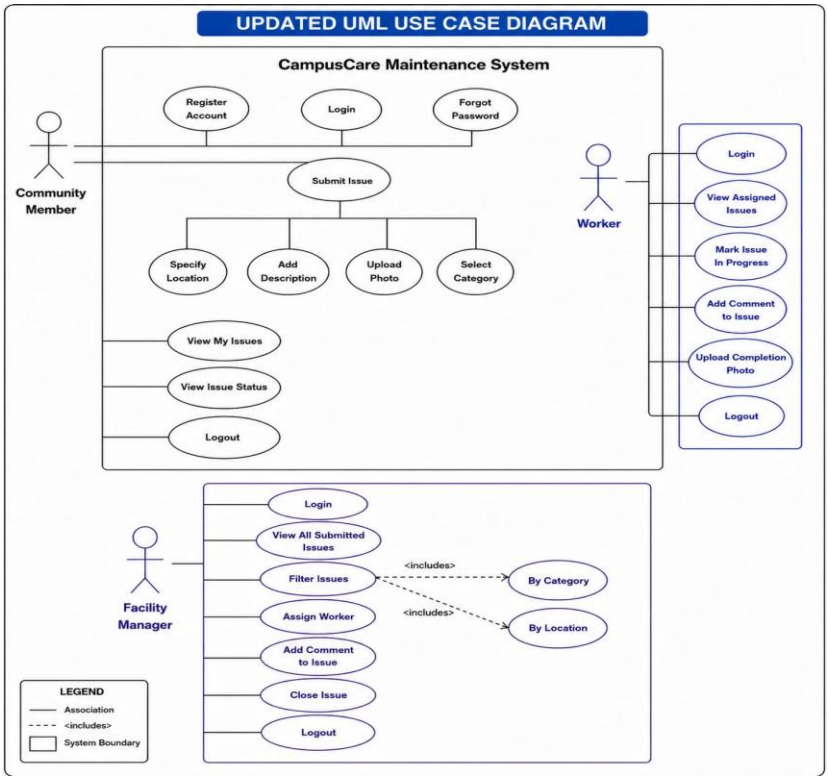
Facility Manager

- As a facility manager, I want to monitor all submitted issues.
- As a facility manager, I want to assign workers.
- As a facility manager, I want to close resolved issues.

5. UML Use-Case Diagram

The UML Use-Case Diagram was updated to reflect the final implemented features of the CampusCare system.

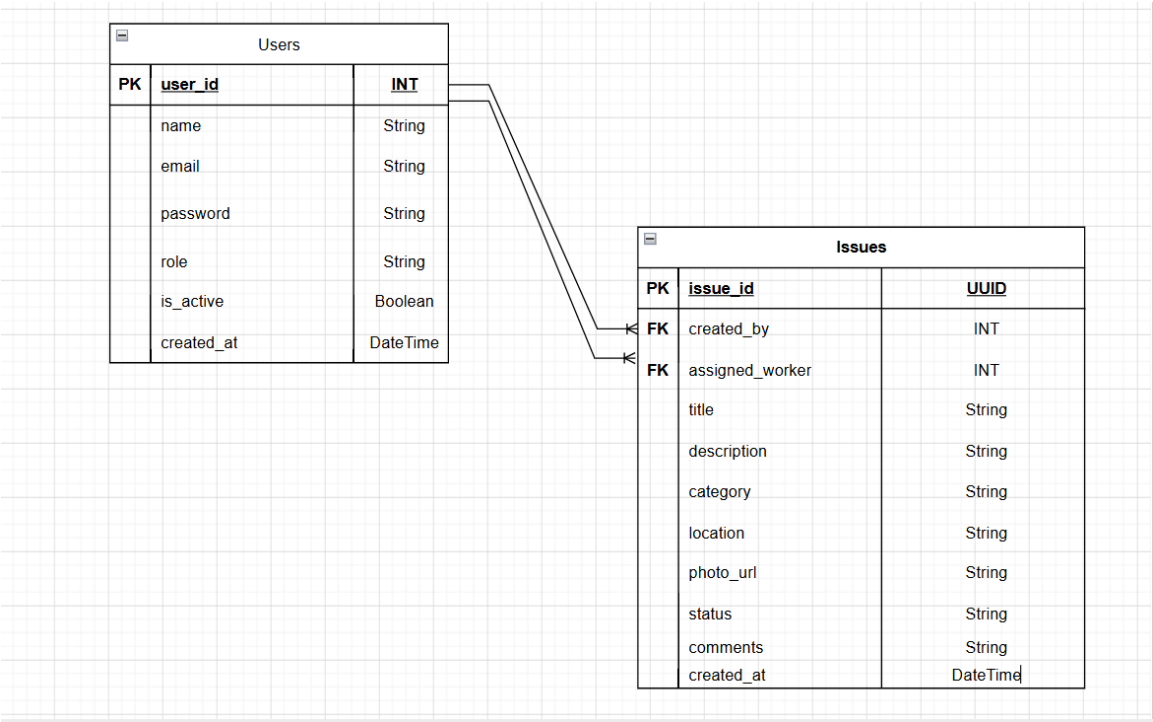
Figure 1: Updated UML Use-Case Diagram



6. Entity Relationship Diagram (ERD)

The ERD was updated to reflect the final database structure used during implementation.

Figure 2: Updated ERD Diagram



7. Non-Functional Requirements

7.1 Performance Requirements

- The system shall load screens within 5 seconds.
- The system shall support multiple concurrent users.

7.2 Security Requirements

- The system shall implement JWT authentication.
- Passwords shall be securely hashed using bcrypt.

7.3 Usability Requirements

- The application shall provide a clean and consistent user interface.
- The application shall support mobile-friendly navigation.

7.4 Reliability Requirements

- Issue data shall be stored immediately after submission.
- Uploaded images shall remain accessible through cloud storage.

7.5 Scalability Requirements

- The architecture shall support future feature expansion.
- The database shall support additional campuses in future versions.

8. Technology Stack

Component	Technology Used
Frontend	React Native (Expo)
Backend	Node.js & Express.js
Database	PostgreSQL (Supabase)
Authentication	JWT Authentication & bcrypt
Storage	Supabase Storage
Version Control	Git & GitHub

9. Database Design

The database contains the following primary tables:

- users
- issues
  - Issue comments and status updates are stored within the issues table.

10. System Architecture Overview

The CampusCare system follows a client-server architecture using React Native frontend, Node.js backend, and PostgreSQL database through Supabase.

11. Implemented Enhancements

- Role-based navigation
- JWT-secured authentication
- Cloud image uploads
- Worker assignment workflow
- Issue filtering
- Responsive mobile UI
- Real-time database updates

12. Conclusion

CampusCare successfully digitizes the university maintenance workflow through a secure and role-based mobile application.

The system improves communication between community members, facility managers, and workers while enhancing issue tracking and operational transparency.